

创始人手册： 打造 AI 原生 创业公司

The Founder's Playbook:
Building an AI-Native Startup



目录 / Contents

第一章 · 2026 重置版的创业生命周期	The startup lifecycle, rebooted for 2026	3
第二章 · 创始人的定义正在改变	What it means to be a founder is changing	6
第三章 · 创意阶段	Idea Stage	11
第四章 · MVP 阶段	MVP Stage	18
第五章 · 上线阶段	Launch Stage	23
第六章 · 扩张阶段	Scale Stage	27
第七章 · 老本行，新规则	Same job, new rules	32
资源	Resources	34

关于本版

本中文版按照 Anthropic 官方手册的章节结构与思想框架编排，重点呈现其方法论、练习与判断标准。关键英文术语与产品名（Claude / Claude Code / Claude Cowork / CLAUDE.md / MCP / TAM-SAM-SOM 等）保留原文。与原版同步阅读时，可对照英文标题快速定位章节。

2026 重置版的 创业生命周期

The startup lifecycle, rebooted for 2026

2026 重置版的创业生命周期

The startup lifecycle, rebooted for 2026

AI 正在重塑创业公司的搭建方式。从未写过一行代码的创始人，今天就能把一款生产级应用推向用户；曾经被当作“励志故事”的“十人独角兽”，正从孤勇的传奇变成有章可循的行动方案。

2026 年，AI 已经能写生产代码、做市场调研、梳理竞争格局、起草投资人材料、跑通运营流程。它把那些曾经让有经验的技术创始人都头疼的工具链、平台与系统集成的学习曲线一笔抹平——更重要的是，它把“谁可以开公司、谁可以做产品”这件事拉回到同一条起跑线。

2026 年，一个好想法能把创始人带得比以往任何时候都远。Agentic coding 把过去需要一整支工程团队才能完成的工作，压缩成一个创始人就能交付的事情。

瓶颈不再是“你能造出什么”，而是“你选择去造什么”。

传统的创业增长曲线大致遵循这样的节奏：验证 → 融资 → 招人 → 开发 → 再融资 → 增长 → 再招人 → 循环。如今，AI 已经擦掉了这条隐含假设——每进入下一个阶段都必须更大的团队、新的技能组合、新一轮融资。

本手册按照新现实，把创业四个核心阶段（Idea、MVP、Launch、Scale）重新画了一遍地图。我们会一一拆开看：当 AI 是你技术与组织建设的底座时，每个阶段长什么样、该用哪些工具、那些在用这些工具的创始人是如何把时间线压扁的。如果你已经准备好规划从想法到出口（exit）的最短路径，请继续往下读。

01	02	03	04
Idea	MVP	Launch	Scale
创意验证	最小可行	公开上线	规模扩张

创始人的定义 正在改变

What it means to be a founder is changing

创始人的定义正在改变

What it means to be a founder is changing

过去，创始人是被"能做什么"这件事定义的：技术型创始人写代码，非技术型创始人跑业务、谈合同。但 2026 年的模型、系统与 AI agent 已经拆掉了这堵墙——"能动手做的人"和"有值得做的想法的人"之间，那道分界线消失了。

AI 原生创业正在从根上改写"创始人"这个角色。没有工程背景的人，也能把脑子里的想法变成跑在生产环境的软件；技术过硬但商业经验薄弱的创始人，也能轻松产出 GTM 策略、财务模型和打磨到位的融资材料。

过去，创始人大部分时间都在"执行模式"里——写代码、管人、应付日常运营。在 AI 原生创业里，创始人这个角色从"亲力亲为的贡献者"，变成"agents 的总指挥"。他指挥的是一批能读文件、跑命令、执行代码甚至上网浏览的专门化 AI 助手。创始人的注意力向上挪了一层，落到更高阶的工作上：产生想法，指挥那些把想法落地的系统（AI agents、工具，以及那一小撮人）。

但 AI 作为核心基础设施带来的最革命性的变化，是它解锁了那些有领域专长、却被技术门槛挡在门外的非技术创始人。当创始人池子的范围从工程师扩展出去，你会看到一批由背景完全不同的人搭起来的公司，他们去解决传统"技术创始人 → VC"的管道里从未优先级、甚至从未注意到的真实问题。

精益创业的三大 AI 杠杆

AI tool capabilities for lean startups

过去的创业模板默认你需要招工程师写代码、招销售卖东西、招运营管公司，人头数被当成组织前进的速度计和产品成熟度的标志。2026 年的早期创业完全是另一种形态——它从设计之初就极度精简，常常就是创始人一个人，加几个伙伴。把技术建设和组织建设都搭在 AI 这个基础设施上之后，他们可以在团队真正扩张之前，先抵达产品验证、早期收入，甚至盈利。

AI 让一家创业公司能像规模大得多的组织一样运转，主要靠三个杠杆：调研、agentic coding、核心业务流程的工作流自动化。

对话式智能与调研 *Conversational intelligence and research*

把它当成你随时在线的全领域专家

把创始人在第一年必须学会、但出发时几乎一无所知的事情列一遍：怎么发工资？产品迭代节奏怎么安排？怎么写一份紧凑的投资人备忘录？

过去，这些问题的标准答案都是"去找一个懂的人问"。对一个 bootstrapped 或 pre-seed 的创始人来说，这要么意味着把本该用来开发的时间花在打听上，要么意味着烧掉一笔早期资金请顾问。现在，他们有了一个随叫随到、覆盖一切想得到的领域的专家。

- **深度调研**：竞品分析、市场规模测算、财务建模
- **文档起草**：融资幻灯片、案例、投资人备忘录、PRD
- **战略思考伙伴**：唱反调（devil's advocate）、事前剖检（pre-mortem）、情景规划、roadmap 优化

Agentic coding 想：那个永远在线、永不卡壳的工程师

过去，要做软件要么得有技术联合创始人，要么得请一家外包开发公司，要么就需要足够长的 runway 先把工程团队招齐，然后才能写下第一行生产代码。

Agentic coding 工具现在让任何一个有想法的创始人，都可以用大白话描述自己想做什么，再指挥 AI 去生成、测试、调试和重构一个生产级的代码库——速度和规模都能赶上一整支工程团队。

"我有个想法"到"我有一个产品"之间的距离被压扁了。创始人的角色现在围着"做什么、为什么做"打转，而 AI 处理那些真给真用户用的、真实的基础设施搭建。

工作流自动化 *Workflow automation*

想：随用随到的自动化运营团队

即便创始人能像咨询顾问一样调研、像工程团队一样写代码，公司里还有一整类工作既不算战略规划也不算产品开发，但还是得有人做。日程、CRM 更新、出周报、文档维护、内容发布、合规跟踪、那些让所有工具和系统能互相对得上话的"结缔组织"……精益创业里，这些活儿主要砸在创始人头上——它实质上在向那些本该用于更高阶决策的时间和注意力收税。

AI 工作流自动化把这笔税免了。重复性运营任务可以配置成自动发生：成交时 CRM 自动更新，周报自己编译自己，产品文档跟着产品变更同步刷新。Claude Cowork 还能把创业公司日常依赖的项目管理工具、沟通栈、数据源都打通——而且不需要专门有人去搭那一堆集成。在 Day Zero 的公司里，那个"专门的人"几乎永远就是创始人本人。

时机和编排是关键

Timing and orchestration are everything

能把 AI 的调研、自动化和 agentic coding 串起来用的创始人，可以让一家公司用远超其人头数的杠杆率运转，并且可以把绝大部分时间和带宽用在真正重要的工作上。

但这件事不是自动驾驶。负责编排这些 AI 工具的创始人，得知道怎么用、什么时候用。本手册接下来的全部内容，就是去拆解走 AI 原生路径的创始人会遇到的目标和挑战，以及如何在每个阶段把 AI 工具高效地用起来。

CHAPTER 3 · 第三章

创意阶段

Idea Stage

创意阶段

Idea Stage

每一个创业者起步时都站在同一个地方：一个挥之不去的问题。这是创意撞上现实的阶段——2026 年的创业要想成功，需要这样一种纪律：在证据还撑不住之前，不去动手做。

这个阶段的工作是调研、用户访谈、竞争分析，以及对反向证据的诚实评估，全部要在你让 Claude Code 写下第一行生产代码之前完成。

本阶段目标

Idea stage goal

创意阶段创始人的核心目标是"调研驱动验证"：在投入资源动手之前，先攒下扎实的证据，证明这个问题真实存在、你的方案真的能解。

实操上，这是一系列必须按顺序回答的问题：

- 这个问题是否真实、具体、频繁到值得围绕它做一家公司？
- 它到底是谁的痛？这群人是否构成一个市场？
- 有没有人在解？怎么解？解得怎么样？
- 一个真正能解决这个问题的方案到底需要做到什么？我的想法做得到吗？

这些问题最终都汇聚到一个终极问题：**这件事到底值不值得做？**

这要求你"动手之前先具体起来"。"大家做费用报销很痛苦"是一句观察；"中型公司里的财务经理每周要花 4 个多小时对账，因为他们用的报销工具跟会计软件不打通"则是一个可被验证的假设。

本阶段出口标准

Idea stage exit criteria

创意阶段的退出条件是找到 problem-solution fit。你建立起了主要来自真实人类对话的定性证据，证明你在为真实存在的人解决一个真实存在的问题——而这一切要在你动手做"那个解决方案"之前。

当你能对下面三个问题都给出肯定回答时，你就可以离开创意阶段：

1. **问题是否真实且具体？** 要肯定回答这一条，你要能精确说出谁会遇到这个问题、多久遇到一次、严重到什么程度，以及他们现在是怎么应付的。
2. **你的方案解决的是不是"那个"问题？** 不是你一开始假设的问题，而是验证过程中浮现出来的那个问题。有时这两者重合，但很多时候并不重合。

3. **你的信号是否足以撑起"开始动手"?** 这个阶段你永远拿不到完全确定的答案，等待"确定"本身就是一种失败模式。但你需要足够多的定性证据，让"做 MVP"这个决定是一个经过推理的判断，而不是一次信仰行为。

本阶段的核心挑战

Idea stage challenges

创意阶段是创业旅程里最关键的一段，因为最致命的错误就发生在这里：在这里跑偏，可以迅速把刚起步的项目带出轨道。这个阶段的大部分挑战都属于同一类：在你的理解还撑不住的时候就跑得太快。所以那些保持深思熟虑、按部就班推进的创始人，会一步一步看到稳定的进展。

把"动手做"当成了"做验证" *Mistaking building for validating*

挑战：当技术不再是拦路虎，一个热血上头的创始人会跳过整个创业旅程里最重要的工作——验证他的想法到底是不是别人需要、愿意用的解决方案。

即便在 agentic coding 出现之前，42% 的创业公司死于"做了没人要的东西"。现在，Claude Code 这类 agentic coding 把"我有想法"到"我有产品"之间的距离压扁到几乎为零，这个失败率只会爬得更高。

对一个有惊艳想法的创始人来说，从来没有比现在更好的时代；但能极速搭出一个看起来像产品的原型，这件事反过来给 AI 原生创业带来一种真正危险的、生死攸关的风险。

很多第一次（甚至有经验的）创始人误以为 AI 让"先验证再开发"这条原则失效，把流程变成了：有想法 → 立刻搭原型 → 把"原型存在"这件事本身当成验证。原型成了"假设原本就对"的理由，而那个假设有没有被真正测试过，根本没人去问。

能跑的原型很容易被误认为"问题确实存在"的硬证据，但它不是。原型真正的用处，是作为跟潜在用户对话时的"压力测试道具"。**这些对话本身，才是真正的证据。**

过早扩张 *Premature scaling*

挑战：当"做"这件事变得毫不费力且即时，你可能会在业务还没要求你这么做的時候，就把执行扩张到远远超前的程度。

过早扩张意味着，在你还没真正验证"这条路值得走"之前，就把自己绑死在某条产品路径上。这一直都是创业杀手，但 AI 让创始人陷入这个陷阱时几乎察觉不到。Agentic coding 助手强大到，你完全可以在没有意识到"自己已经偏离了 problem-solution fit 验证"的情况下，把执行扩张到远超验证之前。

它会用同样的热情去生成、测试、调试、重构一个建在错误前提上的代码库，跟它对待一个绝妙想法时一样卖力。**系统里的智力是你的。**这个阶段的最高指令就是：在"做"这件事如此快、如此不费力的时候，让你的"判断力"始终走在"动手"前面。

失去客观性 *Loss of objectivity*

挑战：问 AI 去找"支持你已经相信的事情"的证据，它一定能找到。确认偏误（confirmation bias）现在配了一台调研引擎。

确认偏误一直是创业者的职业病：创始人天然对自己的想法充满热情。现在，AI 工具给确认偏误装了显著的增益。让 AI 验证你的创业想法，它就给你找出支持证据；让它测算市场规模，它就给你算出一个能让 TAM 看起来可投资的数。

AI 跟着你的方向走——这意味着一个不愿问难题的创始人，可以比以往任何时候都更快地构造一份"做足了功课"的、详尽的、看起来有研究基础的案例，去支持一个糟糕的想法，并且自我感觉良好地以为自己在做尽职调查。

解药是同一个工具，只是反过来用：AI 给一个想法做压力测试的彻底程度，跟它给一个想法找支持证据的彻底程度是一样的。当结构化的对抗思考浮现出"你的想法需要修正"的证据时，那就是该 pivot 的信号。

Claude 如何帮到创意阶段的创始人

How Claude can help Idea stage founders

把一个 AI 原生创业的想法推过 Idea 阶段，感觉上可能会很慢。你是创始人，你只想动手。但这个开局阶段本质上是一次调研与验证练习，所以这个阶段要伸手去拿的工具，是那些能让你在 all-in 写代码之前更严谨地思考的工具。下面是如何在 Claude 的三个产品形态（Chat、Claude Cowork、Claude Code）里使用它来尽快走完 Idea 阶段、同时把尽调做扎实。

Chat、Claude Cowork、Claude Code：怎么选？

AI 让创始人能更快交付、自动化繁琐流程、规模化运营，但你用哪一种界面很重要。按手头任务来选。

Chat——不离开你正在用的那个 app，就能完成的快速对话。适合公司日常里那些不停冒出来的小任务：一句话提炼一份冗长的投资人备忘录、开董事会前对一个数据点做个 sanity check、把一长串 Slack 讨论捋清楚。

Claude Cowork——真正占时间的知识工作。从多个来源里拉信息、做综合、产出一份成品，可能是 doc、deck 或 spreadsheet。比如把一堆客户访谈纪要变成一份主题归纳，给下次产品 review 用；融资前从十几家厂商网站搭一张竞争格局图；或者一个每周一早晨自动跑的任务：从你连接的工具里拉指标，往共享文件夹里扔一份周度 KPI 简报。

Claude Code——给团队里工程师准备的 agentic 编码环境：直接访问代码库、Plan Mode、git 集成、本地/IDE/或沙盒云端环境。精益团队在不断膨胀的代码库上交付新功能、把 MVP 时代的遗留代码迁过去、从原型走到生产，都在这里发生——而且不需要等更多人头到位。

三种形态底下都是同一个 Claude；变的是它周围的工作空间。

定义并压力测试你的问题假设 *Defining and pressure-testing the problem hypothesis*

你的领域 know-how 加上前期调研，已经能产生一个假设。第一份工作是把它磨到真正可被验证的程度。Claude 在这里特别有用，因为它会逼你具体化：到底是谁有这个问题？多频繁？多严重？他们现在是怎么应付的？一个连这些问题都答不清楚的问题陈述，还不到验证阶段。

练习 · EXERCISE 跟 Claude 一起把你的问题陈述磨到"可被验证的假设"为止。例如，"合同审核太慢"基本不可被验证；而"中型公司的内部法务团队每个合同审核周期要花 3 天以上，因为 redline 是在邮件线程里管理的，不是单一版本受控文档"——这个就非常可验证。

下一步是让 Claude 给你的想法唱反调，去找反向证据来反驳你的假设。这种做法能浮现出原本会被"善意综合"悄悄边缘化的负面市场信号、失败的竞品、用户行为模式和结构性障碍。

目标是带着已经被最强反方意见压力测试过的假设进入用户访谈，这样访谈才是真正的开放式探索，而不是寻找认可。

Note · 把 Claude 当作"结构化的反方辩手"用，是整个 AI 创业生命周期里每个阶段都适用的核心用法。

市场调研与绘制竞争格局 *Market research and mapping the competitive landscape*

测一下竞争对手的分量 / Sizing up your competitors

创业圈有个现象叫"对手忽视" (competitor neglect)：因为太专注自己的愿景和执行，系统性地低估同赛道里其他人在做什么。AI 提供了解药：让 Claude 用最有说服力的方式论证，为什么这个赛道里某个竞争对手会赢、而你会输。它会去分析为什么对方的方法其实更好、为什么客户会选他、为什么你以为的差异化壁垒可能根本守不住。

练习 · EXERCISE 让 Claude 按层级画你的竞争格局：直接竞争对手、间接竞争对手、潜在收购方、可能跨界进来的相邻玩家。然后让它论证每一层为什么对你构成真实威胁——不是那种容易反驳的版本，而是真的有威胁的版本。

市场调研 / Market research

Claude Code 可以综合公开的用户反馈，浮现出反复出现的抱怨和未被满足的需求。顺带一个好处：这相当于免费做了一次"竞品用户的定性调研"。

练习 · EXERCISE 让 Claude Cowork 综合你关键来源里的竞品评论，找出现有方案还没解决掉的 top 抱怨。如果你的假设正好命中其中一两个，那是 problem-solution fit 的强信号。如果没命中，那也是值得知道的信息。

Claude Cowork 还可以从厚重的行业报告、分析师文件、市场调研文档里提取相关信息和数字；这些经过清洗和综合的输入，正好成为 Claude 进一步分析的理想 context。

练习 · EXERCISE 用公开数据搭出 TAM / SAM / SOM 模型，压力测试背后的假设。判断这个市场是在扩张、收敛还是成熟期——这个 context 会影响你怎么看时机和差异化。画出买家地图：谁握预算、谁影响决策、这两类人是不是同一个人。

趋势分析 / Trend analysis

最后，让 Claude 帮你听听那些早期信号，看你是否踩在了正确的时间点。盯一下你的问题已经被讨论的 subreddit 和 LinkedIn 群，注意用户描述自己问题时实际使用的措辞。让 Claude 去识别"类似问题被解决过的相邻市场"，把"什么 worked、什么没 worked"提炼出来。把可能加速或威胁这个机会的监管、技术、人口结构趋势浮上水面。

练习 · EXERCISE 让 Claude 找出三个外部趋势（监管、技术、或人口结构），未来两年可能显著影响你的市场，并判断每一个对你这个具体假设到底是顺风还是逆风。

Note · 这一节里的市场调研和竞争格局工作不是一次性练习。你会在 MVP 和 Launch 阶段持续有新发现、持续修正认知，所以每当你的假设演化时，都应该把这些练习重跑一遍。

规划与设计用户访谈 *Plan and design customer discovery*

跟潜在用户聊出来的东西的质量，取决于两件事：（1）你问的问题质量，（2）你问对了人没有。Claude 在用户访谈这件事上特别有用——找谁聊、问什么、以及如何理解你听到的。

找谁聊 / Who to talk to

一份精准的目标画像比一份长长的联系人名单有用得多——具体到职位名称、公司类型、团队结构、最有可能切身感受到这个问题的资历层级。从这里出发，找到这些人实际能被触达的地方——他们聚集的社区、活动、LinkedIn 群、Slack 工作区——然后按"离问题有多近"建立一个优先级框架，决定先联系谁。

问什么 / What to ask

目标定下来之后，用 Claude 来搭访谈框架本身：合适的问题、合适的顺序、设计成浮现"他们实际做了什么"而不是"他们以为自己会做什么"。新手创始人常犯的错是问一个泛泛的、关于未来的开放问题（"你会用这样的东西吗？"），而不是去具体地问相关的过去（"跟我聊聊上一次你处理这个问题是什么时候、怎么处理的。"）。

Claude 可以标出你草拟的问题里哪些是诱导性的、太宽泛、或更可能产生噪声而不是信号的问题。它也能帮你为重要问题里的回避或含糊答复，设计跟进追问。

如果你的假设涉及多个角色画像，Claude 还能为每种画像设计不同的问题组。一个财务经理和一个 CFO 跟同一个问题之间的关系是不一样的，单一访谈框架会把这种差别压平。

练习 · EXERCISE 先自己手写访谈问题，再让 Claude 来审。明确请它标出哪些问题是诱导性的、面向未来的、太宽泛的、或更可能引出社交期望式答复而不是诚实答复的。然后请它对访谈里两到三个最可能出现回避的时刻，建议追问探针。

访谈后分析 / Post-interview analysis

每次对话后，让 Claude 做 debrief：把你的笔记喂给它，请它指出哪些证实了你的假设、哪些挑战了它、哪些真正出乎意料。攒到一批访谈之后，把全部访谈笔记跑过 Claude Cowork，浮现反复出现的主题、矛盾点，以及正反两边最强的信号。然后把这份综合结果带回 Claude，请它标出"你自己对数据的解读"里哪里可能是在 pattern-match 你想听到的东西，而不是真实存在的东西。

练习 · EXERCISE 每做完五个访谈，让 Claude Cowork 综合你的笔记，产出两份清单：支持你假设的证据、挑战你假设的证据。如果第一份明显比第二份长，请 Claude 判断这种不对称反映的是"数据里实际呈现的样子"，还是你"希望看到的样子"。

用户触达与排期 *Customer outreach and scheduling*

用 Claude Cowork 来自动化"建联系人清单、跑触达、约访谈"这件事的运营负担。

Claude Cowork 可以拿你和 Claude 一起定义出来的目标画像（含职位、公司类型、资历层级），调研并整理出一份结构化的潜在客户清单及核验过的联系信息。然后它会按规模起草个性化触达邮件，按每个人的角色和上下文做定制。

当回复进来时，它通过 MCP 接 Gmail 和 Google Calendar，管理对话线程、处理排期请求、把访谈塞进日历。工作流持续推进——Claude Cowork 按设定节奏生成跟进草稿（比如对未回复联系人在第七天的跟进），每步完成时更新你的追踪表，让你随时知道每个 prospect 在 pipeline 里的位置。

练习 · EXERCISE 把你验证过的访谈目标画像给 Claude Cowork，让它建一份 prospect 清单、起草一套个性化触达序列、搭一张追踪表（列：触达状态、跟进节奏、访谈完成情况）。然后让它跑协调工作，你专心准备对话本身。

设计你的最终解决方案概念 *Design your final solution concept*

验证工作完成了：问题真实存在、你知道是谁有这个问题、你有了一个被证据支持的方案概念。用 Claude 从每个角度去发展和挑战你的方案概念：漏洞在哪？有什么替代方案？要让这个方案在 scale 下也成立，得有什么前提为真？

这是一个重要的现实检查点：这个设计真的在解决验证过程里浮现出来的那个问题，而不是你出发时假设的那个问题？

练习 · EXERCISE 把你的方案概念呈给 Claude，请它指出你的设计最依赖的三个假设。然后问每个假设要成立需要什么前提为真、以及如果其中某个不成立后果是什么。

用 Claude Code 搭一个轻量原型 *Build a lightweight prototype with Claude Code*

有趣的部分来了：拿着一个验证过的假设和一个经过压力测试的方案概念，你终于可以动手做点东西了。

这是创意阶段里 Claude Code 真正登场的时刻。即便你一直在小打小闹地折腾，现在才是产出"官方轻量原型"的节点：把你的想法摆到一个真人面前、并拿到真实反应所需的最小表面积。

你不是在做一个真实世界的产品（还没到），你是在搭一个想法的功能性样本，用来跟客户和投资人对话。真实用户对一个他们能真正触碰的东西做出的反应，会告诉你十几次问题-解决方案探索访谈都问不出来的东西。之前你是在确认"你要解决的问题真实存在"；现在，你是在邀请潜在用户跟提议的方案 engage。

练习 · EXERCISE 定义你的方案所依赖的"那一个核心交互"。指挥 Claude Code 只做这一件事。拿到之后，把它摆在你验证过的目标画像里的五个人面前，请他们试用。这五次对话里你学到的东西，决定你是继续做下去，还是回去重画方案。

走到 Idea 阶段的终点，是 AI 创业赛跑里的一次巨大领先——因为现在你押的不是直觉，而是在用证据执行。

下面就进入 MVP 阶段——创始人的核心问题从"这件事值不值得做？"变成"我们应该先做什么？"，而 AI 的主要角色，从"调研伙伴"切换到"施工队"。

MVP 阶段

MVP Stage

MVP 阶段

MVP Stage

很多创始人把 MVP 阶段当成"施工期"，但 MVP 阶段本质上仍然是一次证据收集练习。不同的是，你现在收集的证据是关于"方案"，而不再是关于"问题空间"——具体说，是要证明一个真实、可识别的人群是否觉得这个方案值得用、值得回来、值得付费、值得推荐。

本阶段目标

MVP stage goals

作为 AI 原生创业的创始人，你的目标是把一个已被验证的问题翻译成一个真实用户会真正使用的可工作产品。它不是 roadmap 上每个功能都齐备的完整版本，而是把一个真实方案摆到真实用户面前、产生真实 PMF 证据所需的、最小最聚焦的迭代。

与此同时，**你现在怎么搭，决定了你以后能做什么**。所以 MVP 阶段有第二个同样重要的目标：在不积累那种"会复利、会反咬你"的技术债的前提下尽快推进。等到真实用户大量涌来的那一刻，这类债务会瞬间反扑。

最后，从第一天起就投入"持续的上下文"，是让 AI 持续做你的力量倍增器而不是熵之源的关键。在 AI 原生创业里，你的代码库是你跨多个 session 跟 AI 协作的对象——这让"可读、可解释"成为基础。那些跳过规格说明、架构决策与上下文文件（比如 CLAUDE.md）的创始人会撞上一面可预测的墙：每个新 session 都要重新解释一遍代码库，AI 生成的改动跟最初的愿景越漂越远。

本阶段出口标准

MVP stage exit criteria

MVP 阶段的出口条件是 PMF 的真实证据：可识别的特定人群觉得你的产品有足够价值，去 retain（留下来）、付费、或推荐给别人。

本阶段挑战

MVP stage challenges

MVP 阶段，创始人的最高指令是"速度"和"判断力"。这里的挑战都围绕：你能不能用对的方式、足够快地造对的东西，同时不去抄那些以后会让你付出代价的近路。

Agentic 技术债 *Agentic technical debt*

挑战：因为 AI 实质上把过去那些控制"什么能进生产"的天然瓶颈都拿掉了，所以速度是天然保证的。但当速度成为创始人在 MVP 建设里唯一考虑的变量时，他们就会累积起一种以后很难还清的技术债。

MVP 阶段有一些技术债是合理的，但前提是知道在 scale 之前必须把它管起来。普通技术债是逐步累积、可以慢慢清理或用一个专门 sprint 解决的。**AI 技术债会复利。**

如果没有把规格说明和架构约束写进 AI 能读到的地方，每个 session 都会从头自己推一遍那些基础决策，于是决策开始漂移。最终你得到的代码库背后没有一个连贯的心智模型——不是因为哪个具体的部分糟，而是因为这些部分从来没有被设计成能拼在一起。这个问题真实存在，而且往往是迟来的麻烦。

掉进"假 PMF" *Falling for false product-market fit*

挑战：AI 工具能在早期产出惊艳的数字，但这并不保证市场需要你的产品。

早期势头是创始人最容易心理上飘起来的体验。经过几周或几个月的验证和小翼翼的构建之后，产品发出去那一刻像是在确认你一路都对。Agentic coding 让你抵达这一刻的速度比以往任何时候都快，但**早期 traction 不等于 PMF**。Launch energy 来自一些短暂的力量：创始人的朋友、投资人 portfolio 公司里的潜在买家、一条把流量带飞了的 Hacker News 头条。可惜这些没有一个能可靠地预测六周或十二周之后那波初始推力消退后会发生什么。

零摩擦的范围蔓延 *Zero-friction scope creep*

挑战：当"做"这件事几乎不费力又几乎免费时，永远还有一个酷功能可加、还有一个边角情况可以处理。这种范围蔓延弊大于利。

范围蔓延一直是创业的风险，不同在于过去对它的天然抑制——工程时间的真实成本——在加一个功能只需要一个下午而不是一个 sprint 的时候，已经不再起作用了。

问题在于每一个单独的加法看起来都站得住脚。这个边角情况当然得处理；这个 workflow 用户当然想要。在 agentic coding 里，每加一个东西都几乎不费力，所以当下感觉不像范围蔓延。但当你的产品蔓延出最初的边界时，你就有失去方向感和势头的风险。

解药是**在动手之前写下的范围定义**，描述这个产品做什么、它故意不做什么、以及来自真实用户的具体证据要达到什么样才能 justify 新加东西。这把决策从"我们要不要做这个？"转成了"是不是有足够数量的用户告诉我们没有这个东西他们就没法从这个产品里获得价值？"

因为经验不足而不安全 *Insecure by inexperience*

挑战：那些在还没理解基础安全原则的情况下、就用 AI 工具把应用赶进市场的创始人，最终把用户暴露在本可预防的风险里。

残酷的事实是，agentic coding 工具生成的是"能跑"的代码，不是"天生安全"的代码。功能性的代码很容易——要么这个功能 work 要么不 work。安全漏洞在被利用之前是隐形的，这意味着没有一个天然的反馈循环来提醒一个初次创业的创始人有东西不对劲。把一个能用的 MVP 发给真实用户，就意味着真实数据、真实暴露、真实后果。

轻视安全不是 AI 原生项目独有的问题。各个时代的 bootstrapped 创业公司都常常把安全考虑推到项目末尾，有时候等到生产上线前夕。**在任何用户碰到你的 app 之前做一次安全 review，是把 MVP 推向真实世界的最低责任门槛。**

Claude 如何帮到 MVP 阶段的创始人

How Claude can help MVP stage founders

动手之前先把架构定义清楚 *Define your architecture before you build*

在 Claude Code 写下第一行生产代码之前，先用 Claude 定义并记录下来那些将管控这一阶段所有产出的架构决策：要遵循的模式、要避免的依赖、做了什么 trade-off 以及为什么。这份产出会成为一份聚焦的"架构上下文文档"，并立起 Claude Code 后续作业的护栏。

没有这份上下文，每个 session 都从零开始，Claude Code 被迫自己推断结构性假设。让 Claude Code 在没有护栏的情况下作业，会产出一个能跑、但结构上不连贯的代码库——而在一个不连贯的代码库上迭代和 scale，最终就是浪费时间和 token。早晚会撞到一个不可避免的点：代码塌掉，逼你从头重写。

练习 · EXERCISE 在打开 Claude Code 之前，先打开 Claude，给它描述你在做什么：它解决的核心问题、服务的用户、未来六个月你预期的现实规模。让它帮你定义应当管控你 MVP 建设的架构原则、给定你的约束应当避免的依赖、以及你在这个阶段有意识接受的 trade-off。

接下来，把这份产出存成 **CLAUDE.md** 文件。这是你架构上下文文档的第一个版本，是你建设的第一个产物，也是后续每个 session 都依赖的东西。CLAUDE.md 在 Claude Code 中作为项目级指令，Agent SDK 跑在该目录时会自动读取——功能上就是项目的"持久记忆"。

定义并执行你的 MVP 范围 *Define and enforce your MVP scope*

"没有摩擦的范围蔓延"是 AI 时代 MVP 的标志性失败模式之一。和你定义并记录应用架构一样，你也需要在第一个功能开始造之前定义 MVP 的范围。

Claude 可以帮你写一份"范围文档"，描述你的 MVP 做什么、它故意不做什么、以及"功能修订标准"——要从真实用户身上看到什么具体证据，才能 justify 在此时此刻往里加东西。

当新功能想法浮上来时——它们一定会浮上来——你用 Claude 来压力测试：这究竟是来自用户的真实信号，还是裹了一层"产品思考"外衣的创始人热情？

用 Claude Code 搭你的 MVP *Build your MVP with Claude Code*

架构和范围都定义好之后，Claude Code 成为 MVP 阶段主要的构建工具。用它来生成、测试、调试、迭代你的代码库，但要把每个 session 当作"执行你已经做过的产品决策"，而不是"再扔一些新决策进去"的机会。

每个 Claude Code session 开始时做两件事：（1）重温你的范围文档，（2）把你的 CLAUDE.md 架构上下文文档喂给模型。每个 session 结束时，把这次 session 浮现出的任何决策更新回去。**目标是一个你能讲清楚结构的代码库，而不仅仅是一个能跑的代码库。**

练习 · EXERCISE 给你的 Claude Code 工作建一个简单的 session 模板，里面包含：架构上下文文档、本次 session 的具体任务、要遵循的约束或模式。每次 session 结束，往上下文文档里加一条简短的日志：这次造了什么、做了哪些决策、引入了什么假设。每个 session 五分钟文档化，是抵御架构漂移的廉价保险。

任何用户碰到它之前先做安全 review *Security review before any user touches it*

作为 AI 原生创业的创始人，你的责任是知道你的代码库里有什么、理解任何潜在的暴露面、不把明显的漏洞推给那些把数据托付给你的真实用户。

Claude 可以对 AI 生成的代码做一次有用的"安全首过 review"，能帮你识别常见漏洞。把它做成上线前循环里的一个常规习惯是好事。但它不能替代安全工具，更不能在更高风险下替代人工 reviewer——那些把它当作替代品的创始人，就是会出现在数据泄漏新闻里的那一批。

Claude Code Security 走得更远：它会扫描代码库里的安全漏洞、给出有针对性的补丁建议交人工 review，浮现那些传统方法可能错过的问题。

Note · 本书出版时，Claude Code Security 仍处于有限 beta 阶段，把它纳入工作流前请先确认当前可用性。

练习 · EXERCISE 在部署给任何真实用户之前，把你核心应用代码丢给 Claude 做一次明确目标的 review：身份验证与会话处理、API 响应里的数据暴露、输入校验与注入风险、有已知漏洞的依赖。每条发现都要认真评估是否需要修复，凡涉及身份验证、密钥或数据处理的，都需要人工 review。

上线前搭好你的度量框架 *Build your measurement framework before launch*

那些把"早期 traction"误认作 PMF 的创始人，通常是上线之后才开始追踪数据的人——而且他们用来"衡量"的指标，是用来评估"什么 work 了"的指标，而不是用来浮现"什么没 work"的指标。解药是在第一个用户出现之前就把度量框架立起来。

用 Claude 来定义对你这个产品来说真正重要的指标、benchmark 是多少、数据里出现什么样的模式才算真正的 PMF、什么样只是好看的噪声。具体来说：在你发出 MVP 之前，就把 retention 基线、激活标准、Day 7 和 Day 30 的目标定下来。

接下来，定义对你这个具体产品来说"假阳性"长什么样：只注册不激活、只产生收入不留存、有初始热情但没有重复使用，等等。当数据进来时，让 Claude 给你自己的 traction 做对抗性论证：一个怀疑论者会怎么说这些数字？

管理 discovery 和用户反馈的运营 *Manage discovery and user feedback logistics*

真实用户进入产品之后，运营层快速膨胀。Claude Cowork 处理那些重要但繁琐的工作：建立和维护用户联系人清单、跑触达序列、安排反馈访谈、对 bug 报告分诊、追踪迭代周期。Idea 阶段管理 discovery 运营的那些 MCP 集成在这里同样适用。

在反馈收集循环里保留一个"人"——用来对细腻的用户反馈做有解读力的探索。比如一个用户说"这个很棒，但我希望它还能……"，这需要解读：这是核心需求还是 nice-to-have？是这个客户独有的，还是代表了某个 segment？缺的功能是真问题，还是上游 onboarding 里的某件事？没有工具能替你回答这些问题。

练习 · EXERCISE 把你的 MVP 阶段反馈循环配置给 Claude Cowork：起草发给早期用户的触达、约反馈访谈、为 bug 报告和功能请求设计结构化收集流程、写每周综合。先自己看这份综合；之后，可以再请 Claude 分析信息，捕捉你可能漏掉的重要点。

朝着证据迭代，不是朝着完成度迭代 *Iterate toward evidence, not toward completeness*

MVP 阶段结束的标志是你拿到 PMF 的真实证据，无论产品感觉上是否"完成"了。宣告"已经达到 PMF、可以从 MVP 走向 Launch"最终是一个把创始人直觉跟收集到的证据结合起来的判断练习。但有一些有用的试金石：

- **Sean Ellis 测试**：问活跃用户："如果再也不能用这个产品，你会作何感觉？"如果超过 40% 的人回答"非常失望"，那是一个有意义的 PMF 指标。
- **"努力程度"测试**：PMF 之前，retention 需要不断的人工干预——频繁触达、激励、个人跟进、英雄式的创始人能量。PMF 之后，产品开始自己做这件事。当东西开始"被拉"而不是"被推"时，"努力程度"的这种转变是最清晰的信号之一。

最终，没有单一数据点能确认 PMF——它是一个必须跨越多个迭代周期都成立的模式。

当证据要求 pivot 时就 pivot *Pivot when the evidence demands it*

那万一你做了所有这些工作，还是看起来到不了 PMF 呢？结果不支持你出发的方向，这不是失败，是系统在工作：MVP 阶段就是设计成在你过度押错答案之前浮现出这种信息的。

当数据不支持你当前产品时，用 Claude 一起把数据在告诉你什么读出来。

- **探索其他客户 segment**。也许那些没转化的用户从一开始就不是对的目标。常常对的受众已经在你的数据里，只是被低估了。
- **调整产品的价值主张**。也许受众对，但你的 MVP 没有跟用户产生共鸣。调整 onboarding、messaging、或核心功能强调点，可能在不改"你做了什么"的情况下解决问题。

对"脱节有可能深到需要做更根本性改变"这件事保持开放。

练习 · EXERCISE 如果你已经做了三个或更多迭代周期、但在朝 PMF benchmark 移动这件事上没有有意义的进展，在决定下一步之前用 Claude 跑一次诊断。把你的 retention 数据、用户反馈、原始问题假设喂给它，问它三个问题：

- 数据里有没有某个 segment 的响应模式跟其他人不一样？
- 设计价值与体验价值之间的差距是定位问题还是产品问题？
- 要让当前产品找到真正的 PMF，得有什么前提为真？给定你现在看到的，这个情境现实吗？

让答案决定你是要调整、pivot，还是退回 Idea 阶段。

上线阶段

Launch Stage

上线阶段

Launch Stage

如果 MVP 阶段是要证明"你的产品配存在", 那么 Launch 阶段是要证明"你的业务配增长"。

本阶段目标

Launch stage goals

在 Launch 阶段, 创始人必须把早期 traction 变成一台可重复、可持续的增长引擎。除了把产品做到 production-ready, 你还得把它底下的基础设施加固, 同时围绕产品搭一家真正的公司起来。

在 Idea 和 MVP 阶段, 公司天然以创始人中心——你需要全局的情境感知和紧密的反馈循环。但到了这里, 那些仍然试图亲手抓住每一根线的创始人, 会变成 Launch 阶段的瓶颈。**目标不是把你自己从公司里挪走, 而是搭出能解放你注意力、让你只去处理"只有创始人才能处理的决策"的运营系统。**

本阶段出口标准

Launch stage exit criteria

1. **增长是可重复且来自渠道的。**你不只是在留住用户, 你在通过具体渠道可预测地获取用户——CAC、LTV、回本周期都是你知道并能为之辩护的数字。
2. **产品能扛住生产负载。**基础设施已加固, 安全与合规到位, 可靠性在真实生产条件（而不仅仅是你测过的条件）下扛得住。
3. **运营无需创始人作为瓶颈。**流程已存在、自动化已就位。你不再是亲自处理客服、分诊、sprint 规划或报告的那个人。

本阶段挑战

Launch stage challenges

找到 PMF 是早期创业里最难的问题。现在, 创始人的挑战变成了"留住它"。Launch 阶段是那些找到了真实产品 traction 的公司, 仍然可能因为围绕产品的组织跟不上而崩盘的地方。

技术债到期偿付 *Technical debt comes due*

挑战: 为速度和验证而搭起来的 MVP 代码库, 跑得足以证明产品行得通, 但生产流量、新功能、不断增加的复杂度, 正在把那些近路全部暴露出来。

MVP 时累积一些技术债, 是为速度而做的合理 trade-off。到了 Launch 阶段, 这些债开始计息, 越拖越贵。

解药是系统性的架构 audit 来识别结构性弱点、有针对性的重构来处理最糟的部分、以及有意义的测试覆盖扩展，确保下一轮功能开发不会再把同样的问题引回来。

创始人变成瓶颈 *The founder becomes the bottleneck*

挑战：在 MVP，创始人在每个 loop 里都是资产；到了 Launch，随着支持量增长、产品决策堆叠、运营复杂度倍增，同一个本能变成了约束。

从"做工作"到"设计做工作的系统"是创业生命周期里最难的转变之一。因为很少有一个明确的"发生时刻"，风险就是完全错过它、继续留在 builder 模式而组织在你周围停下来。征兆包括：本来应该一小时的决策现在要等你一周；客户支持请求堆积，只有你知道答案；运营任务只在你亲自记得做的时候才发生。

解药是对一切你亲自处理的东西做一次彻底的 audit——从最小的任务到最重要的决策——以识别什么可以系统化、什么可以委派、什么真正还值得创始人的时间和注意力。

安全和合规不再能往后推 *Security and compliance are no longer deferrable*

挑战：MVP 时把安全合规保持简单还可以，但现在有了真实用户、真实数据、桌上可能还有企业合同，这就成了负债。

解药是在 scale 来之前、而不是之后做一次系统性的安全与合规 review，并把浮现的每一项都当作必须修复的 remediation，而不是建议。

没准备好就扩张 *Expansion before you're ready*

挑战：新市场和新融资机会看起来像是增长机会，但它们也可能是 PMF 死掉的地方。

你已经搭起的初始 traction 是真实的，但它也是针对你早期受众特有的。过早扩张到一个与你原市场显著不同的市场，会引入新的用户行为、合规要求、支付基础设施、和你产品没有围之设计的基线预期。突然之间变量太多，你失去了清晰解读自己数据的能力。同时，你也面临"在追逐新且未被证明的受众时忽视原有用户基础"的风险。

Claude 如何帮到 Launch 阶段的创始人

How Claude can help Launch stage founders

Launch 阶段 Claude 的三种形态都全力投入，且互相支撑：每一个工具的产出都成为另两个的输入。结果会自然复利——一个把三者结合起来用的创始人能拿到大于各部分之和的东西。**这就是超精益创业模型在结构上得以成立的原因。**Claude Code 造产品，Claude Cowork 围绕产品造公司，Claude 把这种产品和组织知识可运营化，一支小团队就能像 n 倍规模的公司那样跑。

在技术债复利之前先 remediate *Remediate technical debt before it compounds*

你的 MVP 代码库能跑，但它也需要一次系统性的 remediation pass，搜出任何可能变成结构性负债的技术债。

首先，用 Claude Code 跑一次完整的架构 audit：识别代码库哪里脆弱、哪些近路会变得维护昂贵、哪里的测试覆盖薄到下一轮功能开发会把同样的问题引回来。把 Claude Code 的 audit 发现反馈给 Claude，做 remediation 工作的分诊和排序：什么必须在下个 release 前修，什么可以等一个 sprint，什么是给定你当前阶段可接受的持续债。

这也是把你 MVP 阶段那些"因为没时间写下来所以只活在你脑子里"的架构决策文档化的时刻。把它们放进 CLAUDE.md，能确保未来每个 Claude Code session 都从一个共同的"系统是怎么设计的、为什么"的理解出发。

练习 · EXERCISE 指挥 Claude Code 对你的 MVP 代码库做 audit，产出一份按优先级排序的清单：结构性弱点、测试覆盖缺口、重构候选。然后把这份清单喂给 Claude，让它跨多个 sprint 给 remediation 工作排序：必须先处理的重大问题、可以与功能开发并行处理的、可以等的。

搭建替代创始人注意力的系统 *Build the systems that replace founder attention*

搭出能解放你注意力、让你只去处理"只有创始人才能处理的责任"的运营系统，要求你精确知道你的注意力在去哪里。用 Claude Cowork 对你当前的运营负载做结构化 audit：记录每一项重复任务、每一个落到你桌上的决策、每一个只因你亲自记得才会发生的工作流。然后让 Claude Cowork 把这份清单分类：可以完全自动化的、需要人但不一定是你的、真正需要创始人判断的。

audit 完成后，用 Claude Cowork 来设计自动化候选的工作流逻辑：什么触发每个 workflow、决策规则是什么、产出长什么样、完成后去哪。

把安全与合规做成产品工作流 *Make security and compliance a product workstream*

用 Claude Code 浮现代码层面的问题，对照 SOC 2、GDPR、HIPAA 等你目标市场所要求的审计与标准。这会同时浮现漏洞和合规缺口。把这些发现喂给 Claude，帮你按优先级排 remediation 工作，并设计企业买家在签约前会要求的控制、审计、日志和访问管理。Note：AI 扫描是辅助，不是合格合规 review 的替代。

然后，把合规工作流嵌入你的开发周期，而不是当作一次性项目跑——合规文档需要持续维护和更新。对那些正在接触企业合同或国际市场的创始人来说，这也是 Claude Code 安全扫描可以帮助你准备独立安全评估的时刻。

练习 · EXERCISE 用 Claude Code 对你目标市场需要的框架做一次代码层面的安全 review。把输出喂给 Claude，让它产出两样东西：一份按优先级排序的安全 remediation 序列；一份"为满足某个潜在企业买家的合规 review 你需要准备的文档和控制"清单。

把你一直在跳过的产品管理流程立起来 *Stand up the product management processes you've been skipping*

Launch 阶段需要一组轻量、可重复、能在没有创始人触发或介入的情况下运转的流程。用 Claude 来设计：你的产品时间线和工作周期如何组织、一份 spec 在 Claude Code 碰这个功能之前必须包含什么、bug 报告如何分诊和路由、你的周指标报告覆盖什么以及如何分发。

流程设计完成后，用 Claude Cowork 搭建并跑运营层：安排 sprint 仪式、把进来的 bug 报告路由到对的地方、从你连接的数据源汇编周指标、维护让用户信号持续流入产品决策的反馈循环。

练习 · EXERCISE 让 Claude 设计一个轻量级的产品管理操作系统：定义好的 sprint 节奏、最小 spec 模板、bug 分诊决策树、从你实际数据源拉数的周指标简报。然后用 Claude Cowork 实现并跑这个系统中那些重复性运营元素（排期、路由、报告汇编），让它们按节奏自动发生，不需要你。

扩张阶段

Scale Stage

扩张阶段

Scale Stage

到了 Scale 阶段，创始人的角色重新调向：从 builder 变成对外的公司高管。产品依然是核心，但你日常的工作越来越关于公司本身。你的注意力必须扩展到 Scale 阶段独有的活动上——分析师 briefing、IPO 路演——同时你还要努力保留以 AI 为中心的精益结构性优势。

本阶段目标

Scale stage goals

技术基础设施的 scale 工作还在持续，现在又加上了把组织本身做 scale、把它发展成一家成熟生意的工作。

Scale 阶段你看的是从千级用户到百万级用户、从一个市场到多个市场。在之前所有阶段，增长是你可以通过贴近用户、根据紧凑反馈循环加上一剂创始人直觉来"凭手感"调出来的东西。现在不同了：目标是搭建由成熟的组织运营所支撑的"系统性增长"。

对一个 AI 原生创业来说，你的目标应该是通过"积累出来的深度"来建立一条可防守的护城河——这个深度来自你建进产品里的专业知识、你产品与用户依赖的其他工具和平台的集成深度、以及专有的系统数据与工作流。那些一直朝同一个方向、在同一个基础设施上持续构建的创始人，现在拥有一些真正难以被复制的东西。

在这个阶段，公开市场投资人、分析师、监管机构、企业采购团队、收购方会施加更大的压力，也带来更多的怀疑——因为现在赌注更大了。你的产品和组织必须扛得住外部 scrutiny：不仅是你造的东西的能力，还有围绕它的治理、合规姿态、财务控制、和战略叙事。

本阶段出口标准

Scale stage exit criteria

Scale 阶段的出口条件不再是一个单一里程碑，而是一个阈值事件：即便创始人越来越不直接管日常运营，公司也是可持续的。你已经证明了系统性增长；建立了能满足最苛刻外部 reviewer 的组织治理和合规基础设施；并且对"如果一个资金充足的在位者今天复制了你的产品，你的用户会留下吗？"这个问题有一个扎实的答案。

实践中，这个阈值通常会以三种形式之一出现：无需外部资本即可持续盈利的规模、IPO-readiness、或被收购。三种都要求你的增长是系统性、可审计的，你的产品护城河经得起 scrutiny，你的组织在运营上成熟且可持续。

当这些都成立时，恭喜你的创业从一个"赌"变成了一门"生意"。

本阶段挑战

Scale stage challenges

把运营层委派出去 *Delegating the operational layer*

挑战：Scale 阶段的运营系统必须能在不被看护的情况下可靠、可持续地运转。对于一个从第一天起就亲力亲为的创始人，这个转变既是结构上的也是心理上的挑战。

你 Launch 阶段的工作是创建系统；到了 Scale 阶段，工作变成（1）把这些系统培育到完全可信任的程度，然后（2）真的去信任它们。这比听起来难得多。即便你是一个善于委派的创始人，"什么该交出去、什么留在桌上"也不总是显而易见的。交得太多太快——尤其是给 AI 自动化系统——关键决策可能在缺少只有创始人才能提供的关键 context 时被做出来。但抱得太紧太久，你又会变成瓶颈。

这里的根本挑战是识别那些只活在创始人脑子里、或活在没文档化的 workflow 里的"机构知识"，然后把它编码进一个文档化、可审计、可转移的系统。

scale 技术运营 *Scaling technical operations*

挑战：客户不再只评估你的产品；他们还想知道你的组织能不能作为可依赖的基础设施伙伴。

前三个阶段的技术挑战都围绕代码库——造对的方案、不积累技术债、为真实用户加固安全与合规。到了 Scale 阶段，挑战变成了围绕代码库的一切：搭建支持基础设施、文档、可靠性保证——这些都是"成熟"的信号。签多年期合同的大型客户和机构买家在签约前会要这些，签了之后会拿这些来要求你。

scale 组织职能 *Scaling organizational functions*

挑战：不管你是几个人在跑，Scale 阶段的公司一般都需要招聘、薪酬、会计、法务运营这类组织基础设施。

在 Launch 阶段，系统化运营意味着把消耗创始人注意力的工作流自动化。到了 Scale 阶段，公司需要再生长出一个更广、某些意义上也更具后果的运营职能阵列——比如财务报告、合规监控、合同管理、客户支持，等等。

搭建 GTM 职能 *Building a GTM function*

挑战：有机增长有上限，大多数 Scale 阶段的创始人在他们不得不真正搭一个 GTM 职能之前就撞到了这个上限。

Idea、MVP、Launch 阶段的增长往往来源于创始人亲自销售——从一条 timing 好的 Product Hunt 帖子，到跟早期客户的私人关系。这种有机增长只到一定程度就到顶了，多数公司在 Scale 阶段撞上这个上限。征兆包括：用户曲线变平、CAC 上升、pipeline 只在创始人亲自介入时才动。

Scale 阶段的增长需要建立一个专门的增长引擎，去触达比早期更广的受众。但大多数创业者可能从来没有跑过 marketing、sales、analyst relations 这类东西。合格的 GTM 不仅要建立新的系统和流程，还要创造关于你产品的品牌声音和故事。

幸运的是，GTM 职能不必规模庞大才能有效——把产品造起来的同一套 AI 基础设施，也能 bootstrap 它的市场化。

Claude 如何帮到 Scale 阶段的创始人

How Claude can help Scale stage founders

把日常任务交给 Claude Cowork *Handing off day-to-day tasks to Claude Cowork*

Scale 阶段开局时，清醒地看清“你最需要把时间和注意力投到哪里”。这对从来没造过生意的初次创业者来说是个挑战。Claude 可以帮你列出“在这个阶段只有你才应该做的事”——比如产品叙事决策、董事会关系、企业大单、创始人对创始人对话。不在这份清单上的，都是委派或 Claude Cowork 自动化的候选。

练习 · EXERCISE 用 Claude 产出你当前运营层的“瓶颈地图”：每一个当前路由经过你的 workflow、决策、批准。然后让 Claude 推算：如果你一周不可用，每一个会变成什么样。卡住的那些 workflow，就是你还亲手到能让进展脱轨的地方。

把技术运营 scale 到企业级基础设施 *Scale technical operations into enterprise-grade infrastructure*

当你 scale 时，买家需要安心：你的产品和组织可以被信任作为长期基础设施。代码库内部的技术工作照常进行，但现在还有围绕代码库的技术工作。

第一步是把“机构知识”转换成可 scale 的系统。用 Claude 起草并维护企业采购期望看到的书面基础设施——产品文档、支持 playbook、SLA。同时指挥 Claude Code 对照企业合同要求的可靠性和安全标准来 audit 并加固代码库；并搭建过去基于 Discord 社区支持时不需要提供的技术支持基础设施：日志、监控、事件响应工具、让 SLA 真正可执行的可观测层。

Claude Cowork 则跑企业支持本身的运营层：工单路由、升级 workflow、由产品变更触发的文档更新、续约追踪、以及企业 customer success 所依赖的报告节奏。这三者合在一起，能让一支小团队呈现远大于自身规模组织的支持姿态——这正是签下一份多年期企业合同要求你展示的能力。

练习 · EXERCISE 挑出你最苛刻的三个 prospect 或最想签的三个理想客户。让 Claude 产出一份 gap analysis：每家公司的企业采购团队在签多年期合同前会期望看到什么样的文档、SLA、支持基础设施，你目前在哪里有差距？用产出来跨 Claude Code 和 Claude Cowork 安排技术和文档工作的顺序。

建一支真正的 GTM 职能 *Build a real GTM function*

创始人的 hustle 带你到了这里，但 scale 创业需要创建并执行真正的 GTM 战略。AI 能帮你建、然后跑整套 GTM 引擎。

Claude 可以协助从零建立 GTM 基础资源：市场细分、信息架构、分析师关系策略、销售 playbook、以及当你开始跟公开市场投资人、企业买家和华尔街分析师对话时所需要的、面向投资人的指标叙事。每种受众都有自己的词汇、按自己的标准评估你；Claude 的工作是把你产品的价值主张翻译成对每一类受众都相关的产品营销路径。

然后 Claude Cowork 可以成为你的战术执行层：内容流水线、外呼序列、分析师 briefing 物流、新闻稿和 PR 节奏、CRM 卫生、pipeline 报告——把 GTM 战略转成真实商业动作的各种循环周期。

当 GTM 需要产品营销基础设施——交互式 demo 环境、集成文档、sandbox 租户、API 参考、技术单页——Claude Code 可以帮你造出来。在 Scale 阶段，一段 Loom 视频加一份 sales deck 已经不够了。这也是让 GTM 动作能异步运行的基础设施：一个搭得好的 demo 环境会在你开董事会时帮你 close deal。

把领域专长和机构知识转成 AI context *Turning domain expertise into AI context*

很多超精益创业者正在为他们亲身经历或在某个领域观察到的真实问题，造高度具体的 app 或工具。Agentic AI 现在让那些从未写过一行代码的创始人，也能用自己的领域专长造出能解决复杂问题的产品。Claude、Claude Code、Claude Cowork 各自在“把创始人知识转换成可复利的产品具体性”这件事上扮演角色。

用 Claude 来捕捉、组织、提炼创始人知识，把领域专长放进产品能触达到的地方。通过持续对话、project 和 memory，创始人可以把他知道的一切——行业行话、监管陷阱、边角情况、用户的挫败感、显而易见的答案为什么不 work——分享进一个结构化、可搜索的 context。Skills 可以把重复出现的 workflow（比如“我怎么审一份商业地产租约”、“我怎么分诊一份病人就诊表”）编码成 Claude 每次都同样方式运行的可复用例程。几个月下来，这会变成一个通用 AI 无法匹敌的专有知识底座。

把你的领域知识外化给 Claude，对于把行业专属边角情况编码进你产品来说至关重要：通用医疗账单工具在 340B 药品项目报销上出错，但你的有针对这类情况的专门逻辑。Claude Code 帮你把同行业其他专家经历的常见挫败，翻译成验证逻辑、prompt 精修、或与竞争对手没听过的小众行业系统的 MCP 集成。结果是你的 app 或工具的深度和广度持续复利，竞争对手根本无法复制。

练习 · EXERCISE 挑一个通用竞品在你这个垂直领域里一定会做错的边角情况。基于你真实见过的场景，用 Claude Code 为它造一个专门的测试用例（不是单元测试）。每当类似的边角情况出现时，就加进去。你的测试套件就会变成一张你护城河的地图。

把积累的用户数据复利成可防守优势 *Compound accumulated user data into a defensible advantage*

当用户跟你的产品交互时，他们产生行为信号（比如他们接受哪些输出、拒绝哪些），这些反过来又喂给产品 roadmap。久而久之，你会学到你这群特定用户的具体模式、偏好、边角情况。这就是我们说的复利价值：每一次改进都让产品更有用，更有用驱动更多使用，更多使用产生更多反馈，更多反馈驱动更多改进。

这种数据是时间锁定的、上下文专属的、对模仿者来说几乎不可能重建：你根本买不到几千个用户在你产品里几个月乃至几年精炼出来的工作流的行为指纹。

练习 · EXERCISE 把你产品交互数据的概览喂给 Claude：你一直在收什么、收了多久、对用户随时使用你产品的方式你知道什么。让它找出这份数据里"信号最强的三个行为模式"，并为每一个设计一个把"持续使用"转成"系统性模型改进"的反馈循环。然后让它帮你起草一份单页的"护城河叙事"用于产品营销：你的数据 flywheel 是怎么转的、转了多久、为什么一个资源充足的对手今天起步两年也复制不出来。

构建工作流锁定 *Create workflow lock-in*

复利的数据网络效应让你的产品更难被复制；用户工作流锁定让你的产品更难被离开。用户在他们的日常运营里跑你的产品越久，它就越深地嵌进他们实际的工作方式里。他们在它上面建了自动化、训练人去用它、把它接到他们的数据源和其他工具。他们开发的 prompt、精炼的 workflow、标准化的产出，都围绕你产品做的事和做的方式被塑造。到了这个点，切换从一个产品决策变成了一个全规模运营项目。

第一步是让 Claude 按集成深度给你当前客户基础画地图。对每个客户 segment，识别他们在你产品之上搭了哪些 workflow、依赖哪些集成。这显示出你产品在哪里粘住了，又在哪里需要更深。

你提供的集成越多，客户在你产品周围构建依赖性 workflow 的表面积就越大。Claude Code 帮你快速搭起跟目标用户依赖的数据管道、项目管理工具和其他系统的原生集成。Claude Code 还能造 API、webhook 和 SDK——让客户不只是使用你的产品，还能在它之上构建。这是最深的一种锁定。

练习 · EXERCISE 让 Claude 帮你为前十大客户做一份工作流集成 audit。对每一家，记录他们建了哪些自动化、依赖哪些集成、有哪些团队 workflow 跑在你产品上、以及你估计的切换成本。然后让 Claude 找出整组客户里的模式：什么类型的集成对你这个具体产品产生最深的锁定，以及为那些目前还在表层的客户你可以造什么或开放什么来加深集成。

老本行，新规则

Same job, new rules

老本行，新规则

Same job, new rules

在 AI 这个时代，创始人的工作本身并没有变：找到一个真实问题、造出能解决它的东西、把它 scale 成一家有分量的公司。变的是抵达这个目标的路径。在 Idea、MVP、Launch、Scale 这四个阶段，AI 把一个个季度压成一周一周。

过去要花几个月的验证周期，现在只要一下午。一个可工作的原型不再要求一个掌握对的技术栈的联合创始人；它只要求一个清晰的问题和几次跟 coding agent 的聚焦 session。上线就绪从临上线前的一阵忙乱，压缩成一条持续工作流。到了 scale，那些过去逼着创业公司提前招人去救火的运营重量，越来越多可以交给 AI——把你的团队解放出来，把注意力花在那些会变成你护城河的判断决策上。

瓶颈不再是"你能造出什么"，而是"你选择去造什么"。

The bottlenecks are no longer what you can build, but what you choose to build.

资源

Resources

用 Claude 一起构建 / Building with Claude

Building AI Agents for Startups —— 介绍创业公司如何用 agent 来减少 scale 过程中对创始人的依赖。

Claude Code docs —— 从初次安装一直带到进阶 agentic 工作流。Pro-tip: 从 "How Claude Code works" 这篇综述开始。

Claude Code best practices —— Anthropic 内部和各工程团队验证过的模式: context 管理、permissions、规划与 verification 工作流。

Using CLAUDE.md files —— 如何为特定代码库配置 Claude Code。MVP 阶段搭建开发环境的创始人必读。

Claude Code power user tips —— Claude Code 团队自己的工作流模式, 包括并行 session 和 verification 循环。

Get started with Claude Cowork —— 团队如何设置 Claude Cowork、用 skill、plugin 等放大它对整家公司的影响。

Tutorials —— claude.com/resources/tutorials, 一份可搜索的具体任务实操指南列表。

创始人故事 / Founder stories

三家 YC 创业公司用 Claude Code 造公司 —— HumanLayer (F24)、Ambral (W25)、Vulcan Technologies (S25) 如何用 Claude 把原型快速推进市场, 并用 agentic coding workflow 扩展 AI 驱动平台。

GC AI —— 创始人用领域专长造了一个 Claude 驱动的法律平台, 贴合 in-house 法务团队的实际工作方式: 公司专属 playbook、跨职能 stakeholder、可变的风险容忍阈值。

Carta Healthcare —— 用 Claude 驱动临床数据抽取平台, 每年处理 22,000 例手术案例, 把数据抽取时间减少 66%。

Anything —— 基于 Claude 和 Agent SDK, 帮助 150 万用户把想法变成可工作的软件产品, 无需写代码——包括一位非技术创始人造出并已经在卖的完整招聘平台。Anything 的 AI agent 接管整个 build 过程, 让 solopreneur 可以专注于自己的领域专长。

Cogent —— 一家应用 AI lab, 造 agent 来自动化关键企业安全任务。这家创业公司用 Claude 作为推理层, 跨完整漏洞生命周期自动化调查、优先级排序和修复。

Airtree —— 用 Claude Cowork 作为运营基础设施的中心, 把过去散在十几个工具和团队里的数据统一起来。一个人用 skill 搭出一个工作流自动化之后, 整个组织都能用它去做那些一直没做完的待办。

Duvo —— 造 AI agent 来跨 ERP、供应商门户、表格、邮件甚至电话, 运行采购、供应链和品类管理流程。完全建在 Claude 上, 使用 Agent SDK 跨多个 workflow 编排。

Zingage —— 为居家护理机构 24/7 自动化运营搭建的 AI agent 平台。用 Claude 的结构化 tool calling 跨一个 EMR 和多个沟通渠道编排，并用 Claude 的上下文推理造出能给出细腻、患者定制结果的 agent，而不是 pattern-match 到最常见答案。

Kindora —— 一位非营利组织高管基于 Claude Sonnet 造的 AI 平台，用来智能匹配慈善机构和资助方。在把几千个匹配筛到几个值得跟进的之后，Kindora 的 MCP connector 让非营利机构可以直接在 Claude 里访问其 prospecting 工具。

Wordsmith —— 由一位“律师转 CTO”的创始人创立，为 in-house 法务团队提供可靠的 AI 法律技术。Claude 是 Wordsmith 合同审查、协议起草和文档审查能力的推理引擎，工程团队也用 Claude Code 来构建和演化平台本身。

创业支持与机会 / Startup support and opportunities

Anthropic Startups Program —— 跟 Anthropic VC 伙伴合作的创业公司可获得免费 API credit、最高级别的公开速率限制、以及独家创始人活动和 workshop 邀请。

Claude community —— 给 builder 的论坛和社区空间。

Live learning resources —— 大会、网研、直播、回放。

claude.ai